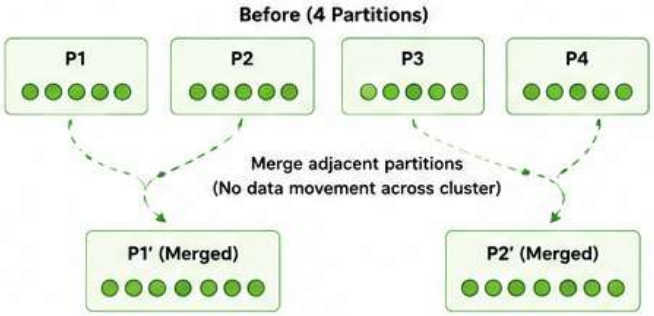
COALESCE vs REPARTITION in SPARK – COMPLETE CONCEPT

Both are used to change the number of partitions in an RDD/DataFrame

1. COALESCE

Used to reduce the number of partitions.
No full data shuffle (just merges partitions).

HOW IT WORKS?

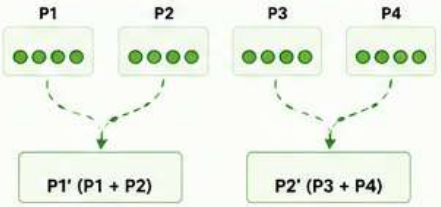


KEY POINTS

- ✓ Only decreases partitions
- ✓ Does not increase partitions
- ✓ No full shuffle
- ✓ Simply merges existing partitions
- ✓ Faster than repartition

EXAMPLES

a) Reduce partitions: 4 → 2
(Using coalesce(2))



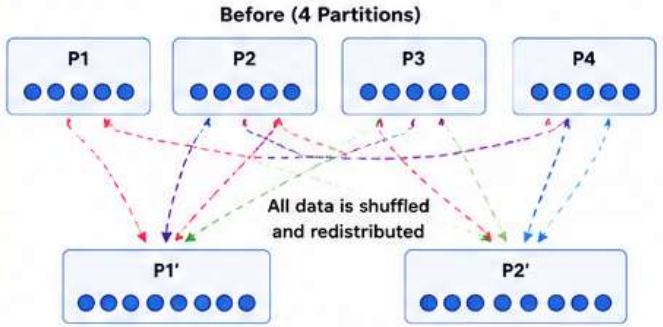
b) Try to increase partitions: 4 → 6
(Using coalesce(6))

Still remains 4 partitions
(Coalesce cannot increase partitions)

2. REPARTITION

Used to increase or decrease the number of partitions.
Performs full data shuffle across the cluster.

HOW IT WORKS?

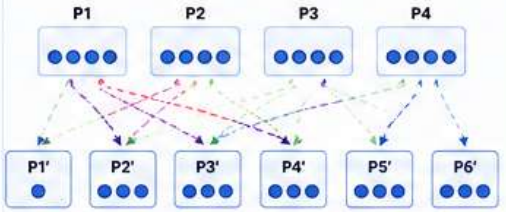


KEY POINTS

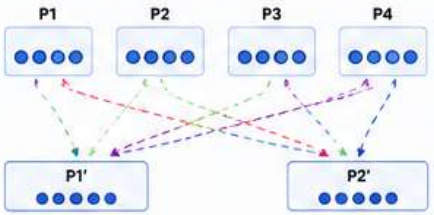
- ✓ Can increase or decrease partitions
- ✓ Performs full shuffle
- ✓ Data is evenly distributed across new partitions
- ✓ Slower than coalesce due to shuffle

EXAMPLES

a) Increase partitions: 4 → 6
(Using repartition(6))



b) Reduce partitions: 4 → 2
(Using repartition(2))



3. WHEN TO USE WHAT?

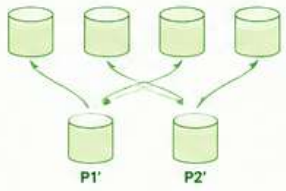
Use COALESCE when:	Use REPARTITION when:
• Your dataset is well balanced (not skewed)	• Your dataset is skewed
• You want fewer partitions	• You want more partitions
• You want to reduce partitions slightly (e.g., 10 → 7, 20 → 10)	• You want to drastically reduce partitions (e.g., 100 → 1)
• You want better performance (avoid shuffle)	• You want even distribution of data

4. COALESCE vs REPARTITION

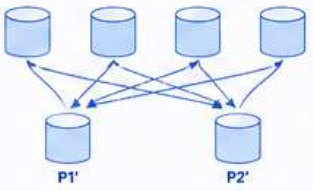
Feature	Coalesce	Repartition
Purpose	Reduce partitions	Increase or decrease partitions
Increase Partitions?	No	Yes
Decrease Partitions?	Yes	Yes
Data Shuffle	No (or minimal)	Yes (full shuffle)
Performance	Faster	Slower
Use Case	Well balanced data	Skewed data / Need more parallelism
Example	df.coalesce(4)	df.repartition(10)

5. QUICK MEMORY TRICK

COALESCE
"Combine without moving"



REPARTITION
"Redistribute by moving"



KEY TAKEAWAY

Coalesce = Merge partitions (Only reduce, No shuffle, Fast)
Repartition = Redistribute partitions (Increase/Decrease, Full shuffle, Balanced)

INTERVIEW QUESTIONS

- What is coalesce in Spark?
- What is repartition in Spark?
- Can coalesce increase partitions?
- When should we use repartition?
- Which is faster, coalesce or repartition?
- What happens if we repartition using column?

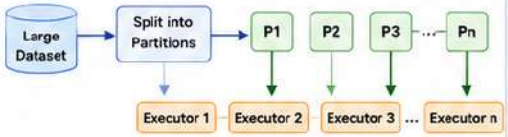
3

SPARK PARTITIONING – COMPLETE CONCEPT

Spark splits large data into smaller parts called partitions and distributes them across executors to run in parallel.

1 WHAT IS PARTITIONING?

Partitioning is the process of dividing a large dataset into smaller parts (partitions) so that Spark can process them in parallel on multiple executors.



WHY DO WE NEED IT?

- ✓ Enables parallel processing
- ✓ Reduces memory requirement per node
- ✓ Improves performance and scalability

2 PROPERTIES OF PARTITIONS

Never span multiple nodes
All the data in a partition is stored on a single node.

One node can have one or more partitions
A node (executor) can store and compute multiple partitions.

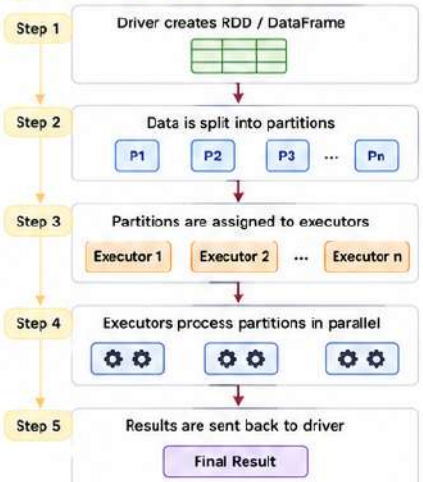
One partition is processed by one core
A core processes one partition at a time.

Number of partitions is configurable
By default, it equals the total number of cores on all available executor nodes.

EXAMPLE (DEFAULT PARTITIONS)

Total Cores in Cluster = 8 → Default Partitions = 8

3 HOW PARTITIONING WORKS?



TYPES OF PARTITIONING

4 HASH PARTITIONING

Distributes data across partitions based on the hash value of the key.
Same key → Same partition.

HOW IT WORKS?

`Partition = hash(key) % number_of_partitions`

EXAMPLE

Number of Partitions = 3

Data (Key, Value)

```
(0, 'A')
(1, 'B')
(2, 'C')
(3, 'D')
(0, 'E')
(1, 'F')
```

Calculation (Partition = key % 3)

```
0 % 3 = 0
1 % 3 = 1
2 % 3 = 2
3 % 3 = 0
0 % 3 = 0
1 % 3 = 1
```

Resulting Partitions

- Partition 0: (0, 'A'), (3, 'D'), (0, 'E')
- Partition 1: (1, 'B'), (1, 'F')
- Partition 2: (2, 'C')

PROBLEMS WITH HASH PARTITIONER

1. DATA SKEW

Some partitions may have much more data than others.

P0 has 5x more data → takes longer → All other partitions wait → Stage delayed

2. SCHEDULING PROBLEM

Too few partitions compared to executors → some executors become idle.

Example:
2 Executors, 3 Partitions

Executor 1 has extra partition → takes more time → Executor 2 is idle.

5 RANGE PARTITIONING

Divides data into partitions based on the range of key values.
Similar values are placed in the same partition.

HOW IT WORKS?

1. Find min and max of keys
2. Divide key space into ranges
3. Assign each key to a range
4. Tuples in the same range go to the same partition

EXAMPLE

Data (Key, Value)

```
10, 15, 20, 35, 40,
50, 65, 70, 80
```

Number of Partitions = 3

Min = 10, Max = 80

Range Boundaries (Equal Ranges)
10 – 33
34 – 56
57 – 80

Resulting Partitions

Partition 1 (10 – 33)	Partition 2 (34 – 56)	Partition 3 (57 – 80)
10, 15, 20	35, 40, 50	65, 70, 80

ADVANTAGES

- ✓ Similar values are grouped together.
- ✓ Better for range queries, sorting, and time-series data.
- ✓ Usually more balanced than hash partitioning (if data is uniform).

6 HASH PARTITIONING vs RANGE PARTITIONING

Feature	Hash Partitioning	Range Partitioning
Based On	Hash of key	Range of key values
Formula	hash(key) % numPartitions	Range boundaries
Same key in same partition?	Yes	Not necessary
Similar values together?	No	Yes
Load balancing	May cause skew	Usually better (if ranges are balanced)
Best suited for	Joins, groupByKey, reduceByKey	Sorting, range queries, analytics on ordered data

MEMORY TRICK

Hash Partitioning
Same Key → Same Partition
(Like lockers with key)

Range Partitioning
Similar Values → Same Partition
(Like numbers in consecutive rooms)

KEY TAKEAWAYS

- Spark splits data into partitions to process in parallel.
- Default partitions = Total number of cores in the cluster.
- Hash partitioning uses hash(key) % partitions.
- Range partitioning uses value ranges.
- Avoid data skew and ensure enough partitions for better resource utilization.

SHARED VARIABLES IN SPARK – IMPLEMENTATION

Two types: 1) Broadcast Variables (Driver → Executors, Read-Only) 2) Accumulators (Executors → Driver, Write-Only)

1 BROADCAST VARIABLES

WHAT IS IT?

Broadcast Variables allow the driver to send a large read-only variable to all executors only once. Executors cache it and reuse it for all tasks.

USE CASE

- Lookup tables
- Configuration data
- Dictionaries
- Mapping data

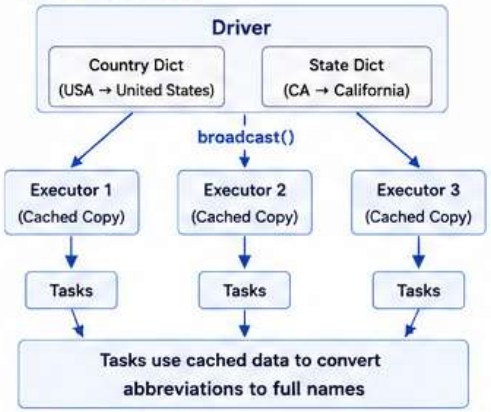
EXAMPLE PROBLEM

Convert abbreviated country and state names to their full forms.

First Name	Last Name	Country	State
James	Smith	USA	CA
Maria	Jones	USA	NY
Robert	Brown	IND	AP

First Name	Last Name	Country	State
James	Smith	United States	California
Maria	Jones	United States	New York
Robert	Brown	India	Andhra Pradesh

EXECUTION FLOW



CODE IMPLEMENTATION

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("BroadcastExample").getOrCreate()
sc = spark.sparkContext

# Sample data
data = [("James", "Smith", "USA", "CA"),
    ("Maria", "Jones", "USA", "NY"),
    ("Robert", "Brown", "IND", "AP"),
    ("Priya", "Sharma", "IND", "KA")]
rdd = sc.parallelize(data)

# Dictionaries
country_dict = {"USA":"United States", "IND":"India"}
state_dict   = {"CA":"California","NY":"New York","AP":"Andhra Pradesh","KA":"Karnataka"}

# Broadcast variables
bc_countries = sc.broadcast(country_dict)
bc_states    = sc.broadcast(state_dict)

# Replace abbreviated names with full names
result = rdd.map(lambda x: (x[0], x[1], bc_countries.value[x[2]], bc_states.value[x[3]]))
result.collect()
```

OUTPUT

```
[('James', 'Smith',
  'United States','California'),
('Maria', 'Jones',
  'United States', 'New York'),
('Robert', 'Brown',
  'India', 'Andhra Pradesh'),
('Priya', 'Sharma',
  'India', 'Karnataka')]
```

KEY TAKEAWAYS

- Sent from Driver to Executors only once.
- Cached on each executor.
- Access using `.value`.
- Read-only – cannot be modified.

MEMORY TRICK

Broadcast = Send once,
Read many (on all executors)

2 ACCUMULATORS

WHAT IS IT?

Accumulators are variables that executors can update (through associative operations like add), but the driver can only read the final result.

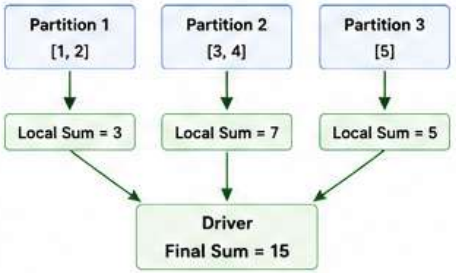
USE CASE

- Counters (errors, warnings)
- Sums and statistics
- Monitoring job progress

EXAMPLE 1 – SUM OF NUMBERS

We want to find the sum of numbers: [1, 2, 3, 4, 5]

PARTITIONS



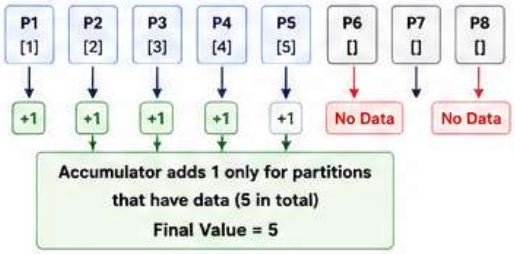
CODE IMPLEMENTATION

```
sc = spark.sparkContext
acc_sum = sc.accumulator(0)   # Initialize accumulator with 0
rdd = sc.parallelize([1,2,3,4,5], 3)

def add_func(x):
    global acc_sum      # Make accumulator accessible
    acc_sum += x        # Add each element to accumulator

rdd.foreach(add_func)   # Run function on each partition
print(acc_sum.value)    # Output: 15
```

EXAMPLE 2 – EMPTY PARTITIONS

RDD with 5 numbers and 8 partitions

PARTITIONS

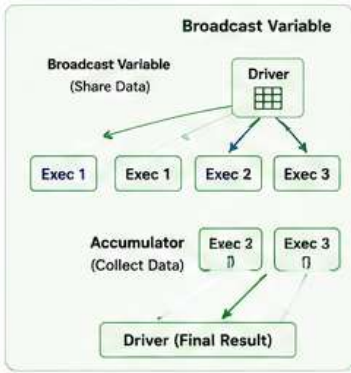


CODE

```
acc_count = sc.accumulator(0)
rdd2 = sc.parallelize([1,2,3,4,5], 8)

print(rdd2.getNumPartitions())   # 8
print(rdd2.glom().map(len).collect())
    # Example Output: [1,1,1,1,1,0,0,0]

rdd2.foreach(lambda x: acc_count.add(1))
print(acc_count.value)      # Output: 5
```

BROADCAST VARIABLES vs ACCUMULATORS

Feature	Broadcast Variable	Accumulator
Purpose	Share read-only data	Collect values from executors
Direction	Driver → Executors	Executors → Driver
Read	Driver & Executors	Driver only
Write	Driver only (before broadcast)	Executors only (add/update)
Mutable?	No (Read-Only)	Yes (through addition)
Common Use	Lookup tables, configs, dictionaries, mapping data	Counters, sums, statistics, monitoring



SHARED VARIABLES IN SPARK – 3 KEY CONCEPTS

1. Broadcast Variable | 2. Accumulator | 3. Comparison (Memory Trick)

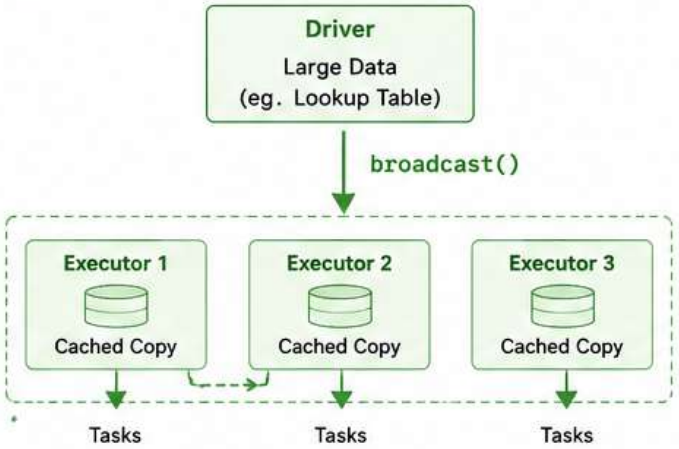
1 BROADCAST VARIABLE

(Driver to Executors – Read Only)

WHAT IS IT?

A **read-only** variable that is sent by the driver **once** to all executors and cached there.

HOW IT WORKS?



SYNTAX & EXAMPLE

```
bc = sc.broadcast(data)
Use: bc.value
```

```
countries = ["India","USA","UK"]
bc = sc.broadcast(countries)
rdd.filter(lambda x: x.country in bc.value)
```

WHY NEEDED?

- ✓ Avoids sending same data with every task
- ✓ Saves network bandwidth
- ✓ Faster execution
- ✓ Best for large static data

KEY POINTS

- Read-only
- Cached on each executor
- Use `.value` to access
- Driver distributes, Executors read

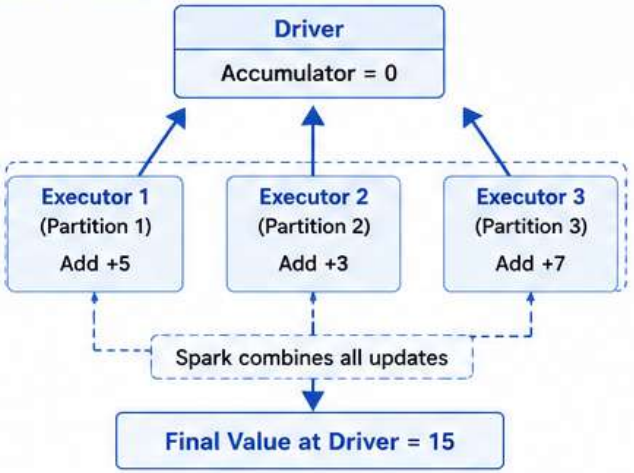
2 ACCUMULATOR

(Executors to Driver – Write Only)

WHAT IS IT?

A **write-only** variable that executors can update **(add values)** and the driver can read the final result.

HOW IT WORKS?



SYNTAX & EXAMPLE

```
acc = sc.accumulator(0)
def add_num(x):
  global acc
  acc += x

rdd = sc.parallelize([1,2,3,4])
rdd.foreach(add_num)

print(acc.value)    # Output: 10
```

WHY NEEDED?

- ✓ Collect counts, sums, statistics
- ✓ Track errors, warnings, processed records
- ✓ Simple and efficient way to aggregate results

KEY POINTS

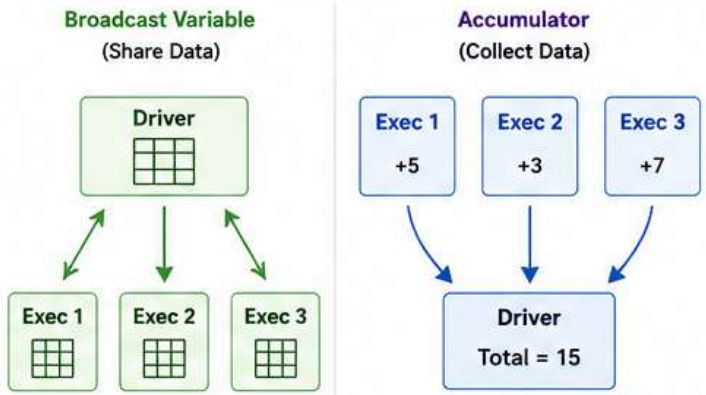
- Executors can add/update
- Driver can only read (acc.value)
- Best for associative operations (sum, count, min, max)

3 COMPARISON & MEMORY TRICK

COMPARE

Feature	Broadcast Variable	Accumulator
Purpose	Share read-only data	Collect values (Counts/Sums)
Direction	Driver → Executors	Executors → Driver
Read	Driver & Executors	Driver only
Write	Driver (before broadcast)	Executors (add/update)
Mutable?	No (Read Only)	Yes (through addition)
Common Use	Lookup tables, configs, dictionaries, reference data	Counters, sums, job statistics

MEMORY TRICK



Broadcast = Send data once to all
Accumulator = Collect results from all